

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

MANYA KULSHRESHTHA (1BF24CS169)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **MANYA KULSHRESHTHA(1BF24CS169)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-6
2	Implement Johnson Trotter algorithm to generate permutations.	8-10
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	11-14
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	15-18
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	19-20
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	21-23
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	24-27
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	28-32
9	Implement Fractional Knapsack using Greedy technique.	33-36
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	37-38
11	Implement "N-Queens Problem" using Backtracking.	39-40

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

CODE:

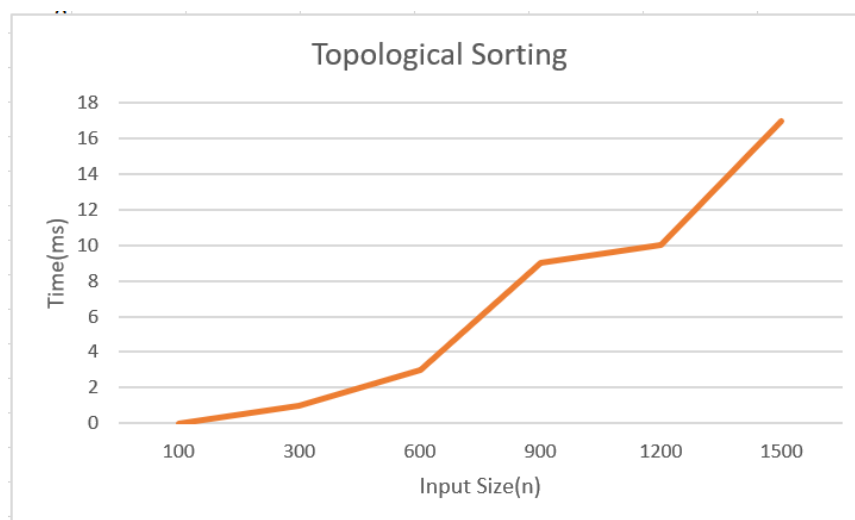
```
#include <stdio.h>
#define MAX 100
int graph[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int n;
void dfs(int v){
    visited[v] = 1;
    for (int i = 0; i < n; i++){
        if (graph[v][i] == 1 && !visited[i]) {
            dfs(i);
        }
    }
    stack[++top] = v;
}
void topologicalSort(){
    for (int i = 0; i < n; i++) {
        if (!visited[i]){
            dfs(i);
        }
    }
}

printf("\nTopological Ordering:\n");
while (top != -1) {
    printf("%d ", stack[top--]);
}
}
int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++){
            scanf("%d", &graph[i][j]);
        }
    }
    topologicalSort();
    return 0;
}
```

OUTPUT:

```
PS C:\Users\kulsu> . & 'c:\Users\kulsu\.vscode
Enter number of vertices: 6
Enter adjacency matrix:
0
1
1
0
0
0
0
0
0
1
0
0
0
0
0
1
1
0
0
0
0
0
0
0
0
1
0
0
0
0
0
1
0
0
0
0
0
0
0
Topological Ordering:
0 2 4 1 3 5
```

GRAPH:



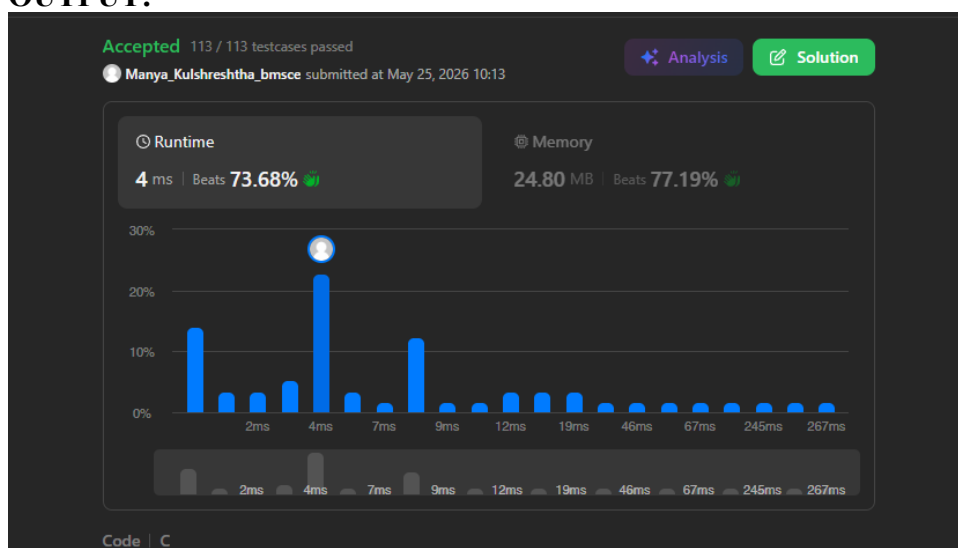
LEETCODE 802. Find Eventual Safe States

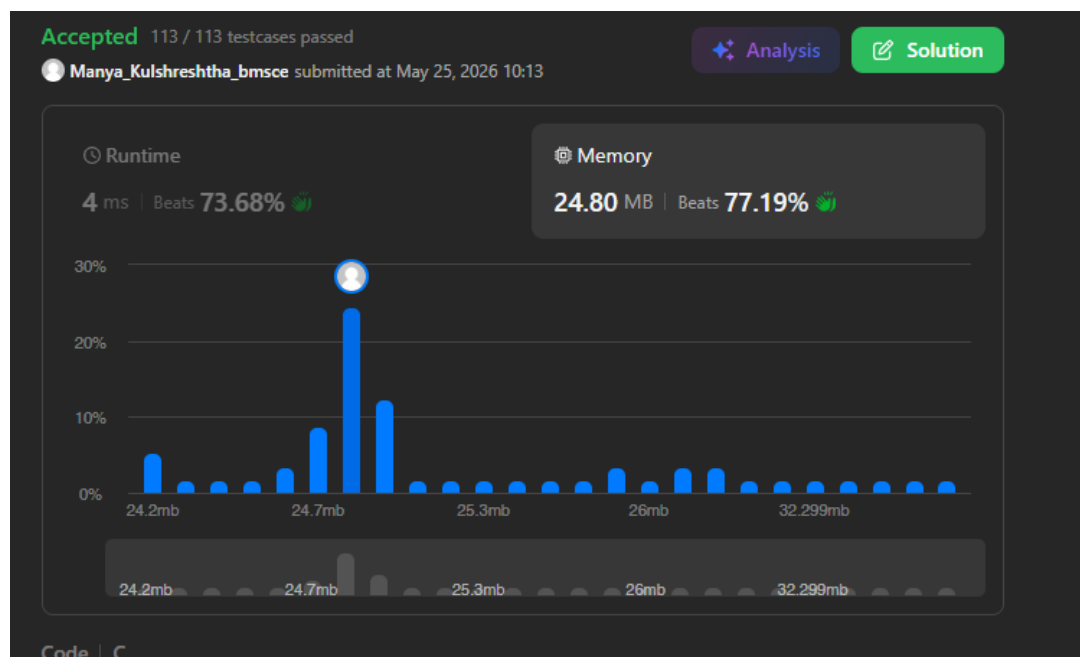
CODE:

```
bool dfs(int node, int** graph, int* graphColSize, int* state)
{
    if (state[node] == 1)
        return false;
    if (state[node] == 2)
        return true;
    state[node] = 1;
    for (int i = 0; i < graphColSize[node]; i++)
    {
        int neighbor = graph[node][i];
        if (!dfs(neighbor, graph, graphColSize, state))
            return false;
    }
    state[node] = 2;
    return true;
}

int* eventualSafeNodes(int** graph, int graphSize,
                      int* graphColSize, int* returnSize)
{
    int* state = (int*)calloc(graphSize, sizeof(int));
    int* result = (int*)malloc(graphSize * sizeof(int));
    int count = 0;
    for (int i = 0; i < graphSize; i++)
    {
        if (dfs(i, graph, graphColSize, state))
        {
            result[count++] = i;
        }
    }
    *returnSize = count;
    return result;
}
```

OUTPUT:





Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

```
graph =  
[[1,2], [2,3], [5], [0], [5], [], []]
```

Output

```
[2,4,5,6]
```

Expected

```
[2,4,5,6]
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

```
graph =  
[[1,2,3,4], [1,2], [3,4], [0,4], []]
```

Output

```
[4]
```

Expected

```
[4]
```

Lab program 2:

CODE:

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1
void printPermutation(int a[], int n){
    for(int i = 0; i < n; i++){
        printf("%d ", a[i]);
        printf("\n");
    }

int getMobile(int a[], int dir[], int n)
{
    int mobile = 0, mobile_pos = -1;
    for(int i = 0; i < n; i++){
        if(dir[a[i]-1] == LEFT && i != 0) {
            if(a[i] > a[i-1] && a[i] > mobile)
            {
                mobile = a[i];
                mobile_pos = i;
            }
        }

        if(dir[a[i]-1] == RIGHT && i != n-1)
        {
            if(a[i] > a[i+1] && a[i] > mobile)
            {
                mobile = a[i];
                mobile_pos = i;
            }
        }
    }
    return mobile;
}

void generatePermutations(int n)
{
    int a[n], dir[n];
    for(int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(a, n);
    while(1)
    {
        int mobile = getMobile(a, dir, n);
        if(mobile == 0)
            break;

        int pos;
        for(pos = 0; pos < n; pos++)
```



```

        if(a[pos] == mobile)
            break;

    if(dir[mobile-1] == LEFT)
    {
        int temp = a[pos];
        a[pos] = a[pos-1];
        a[pos-1] = temp;
        pos--;
    }
    else
    {
        int temp = a[pos];
        a[pos] = a[pos+1];
        a[pos+1] = temp;
        pos++;
    }

    for(int i = 0; i < n; i++)
    {
        if(a[i] > mobile)
            dir[a[i]-1] *= -1;
    }

    printPermutation(a, n);
}

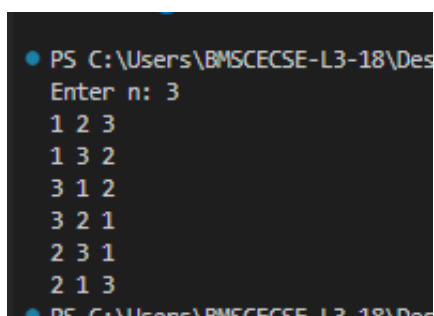
int main()
{
    int n;
    printf("Enter n: ");
    scanf("%d", &n);

    generatePermutations(n);

    return 0;
}

```

OUTPUT:



```

PS C:\Users\BMSCECSE-L3-18\Desktop>
Enter n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
PS C:\Users\BMSCECSE-L3-18\Desktop>

```

```
1 2 3
PS C:\Users\BMSCECSE-L3-18\Desktop>
Enter n: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
PS C:\Users\BMSCECSE-L3-18\Desktop>
```

Lab program 3:

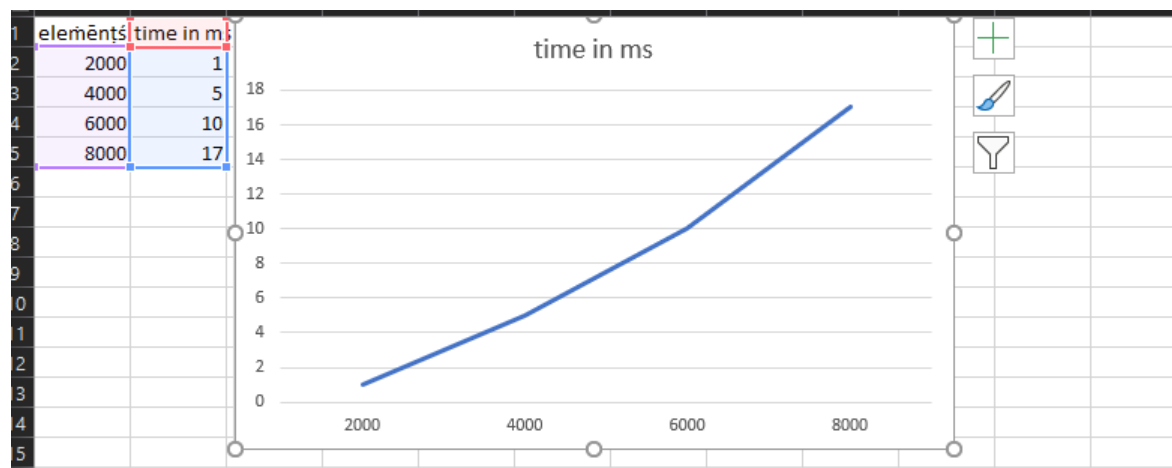
CODE:

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
int c[100000];
void merge(int a[],int l,int m, int h){
    int i=l,j=m+1,k=l;
    while(i<=m && j<=h){
        if(a[i]<a[j]){
            c[k++]=a[i++];
        }
        else{
            c[k++]=a[j++];
        }
    }
    while(i<=m){
        c[k++]=a[i++];
    }
    while(j<=h){
        c[k++]=a[j++];
    }
    for(i=0;i<=h;i++){
        a[i]=c[i];
    }
}
void mergeSort(int a[],int l, int h){
    int m;
    if(l<h){
        m=(l+h)/2;
        mergeSort(a,l,m);
        mergeSort(a,m+1,h);
        merge(a,l,m,h);
    }
}
int main(){
    int n;
    clock_t start, end;
    printf("enter no. of elements\n");
    scanf("%d",&n);
    int *a = (int *)malloc(n * sizeof(int));
    srand(time(NULL));
    for(int i=0;i<n;i++){
        a[i] = rand() % 100000;
    }
    start=clock();
    mergeSort(a,0,n-1);
    end=clock();
    double time=(double)(end-start)/CLOCKS_PER_SEC;
    printf("time taken %f",time);
    free(a);
    return 0;
}
```

OUTPUT:

```
--pid=Microsoft-MEEngine-PID-1q5JrQog.n21 --dbgExe=C:\Program Files\CodeBlocks\MinGW\bin\gdb.exe --1
enter no. of elements
6
enter element
3
enter element
6
enter element
8
enter element
4
enter element
2
enter element
1
1      2      3      4      6      8      time taken 0.000000
```

GRAPH:



LEETCODE 912.Sort an Array

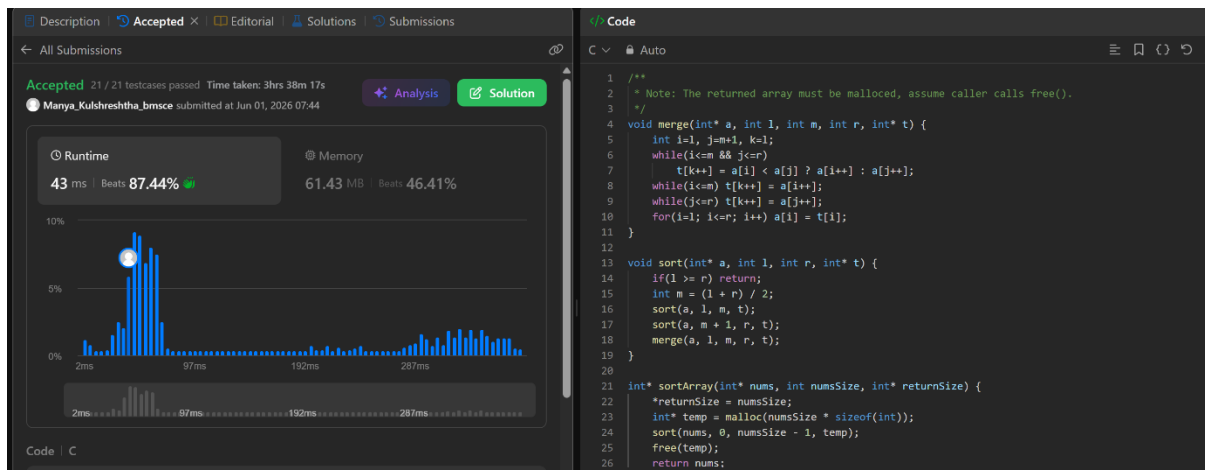
CODE:

```
void merge(int* a, int l, int m, int r, int* t) {
    int i=l, j=m+1, k=l;
    while(i<=m && j<=r)
        t[k++] = a[i] < a[j] ? a[i++] : a[j++];
    while(i<=m) t[k++] = a[i++];
    while(j<=r) t[k++] = a[j++];
    for(i=l; i<=r; i++) a[i] = t[i];
}
```

```
void sort(int* a, int l, int r, int* t) {
    if(l >= r) return;
    int m = (l + r) / 2;
    sort(a, l, m, t);
    sort(a, m + 1, r, t);
    merge(a, l, m, r, t);
}
```

```
int* sortArray(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;
    int* temp = malloc(numsSize * sizeof(int));
    sort(nums, 0, numsSize - 1, temp);
    free(temp);
    return nums;
}
```

OUTPUT:



☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[5,2,3,1]

Output

[1,2,3,5]

Expected

[1,2,3,5]

[♥ Contribute a testcase](#)

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[5,1,1,2,0,0]

Output

[0,0,1,1,2,5]

Expected

[0,0,1,1,2,5]

[♥ Contribute a testcas](#)

Lab program 4:

CODE:

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
void swap(int *a, int *b){
    int temp=*a;
    *a=*b;
    *b=temp;
}
int partition(int arr[],int low,int high){
    int pivot=arr[low];
    int i=low;
    int j=high+1;

    while(1){
        do{
            i++;
        }while(i<=high && arr[i]<pivot);
        do{
            j--;
        }while(arr[j]>pivot);
        if(i>j){
            break;
        }
        swap(&arr[i],&arr[j]);
    }
    swap(&arr[low],&arr[j]);
    return j;
}
void quickSort(int arr[],int low, int high){
    if(low<high){
        int j=partition(arr,low,high);
        quickSort(arr,low,j-1);
        quickSort(arr,j+1,high);
    }
}
int main(){
    int n;
    printf("enter no. of elements\n");
    scanf("%d",&n);
    int *arr = (int *)malloc(n * sizeof(int));
    srand(time(NULL));
    for(int i=0;i<n;i++){
        arr[i] = rand() % 100000;
    }
    clock_t start=clock();
    quickSort(arr,0,n-1);
    clock_t end =clock();
```

```

double time=(double)(end-start)/CLOCKS_PER_SEC;
//for(int i=0;i<n;i++){
// printf("%d\t",arr[i]);
//}
printf("time taken %f",time);
free(arr);
return 0;
}

```

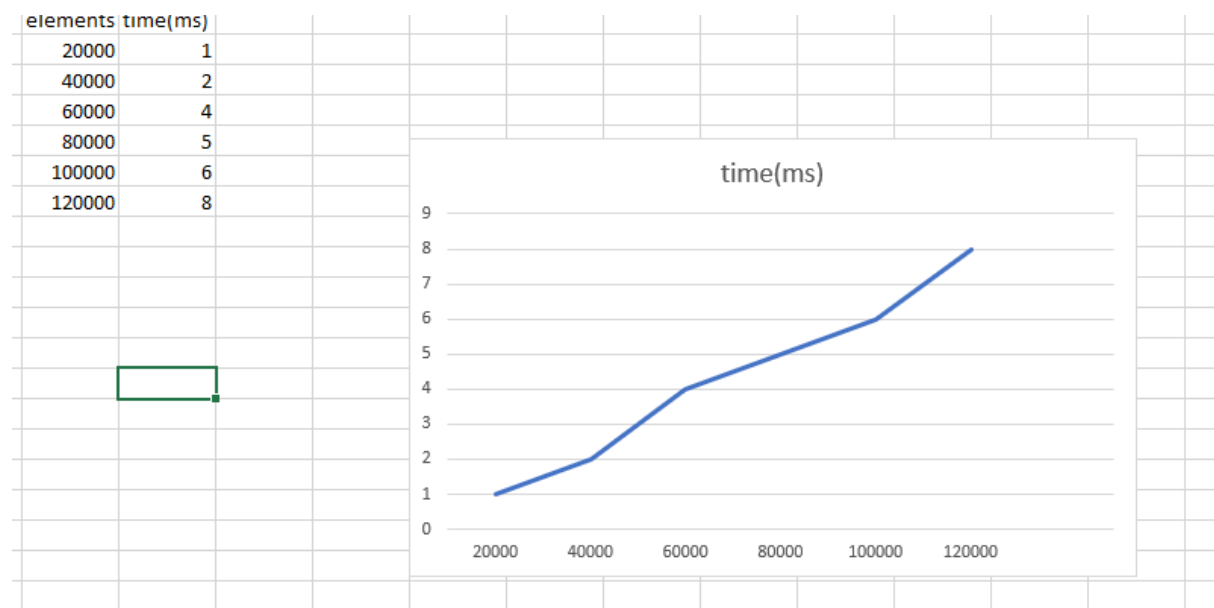
OUTPUT:

```

Enter number of elements:6
Enter elements:1 4 3 6 8 9
Sorted array:1 3 4 6 8 9
Execution time:0.00000 ms
Process returned 26 (0x1A)   execution time : 10.832 s
Press any key to continue.

```

GRAPH:

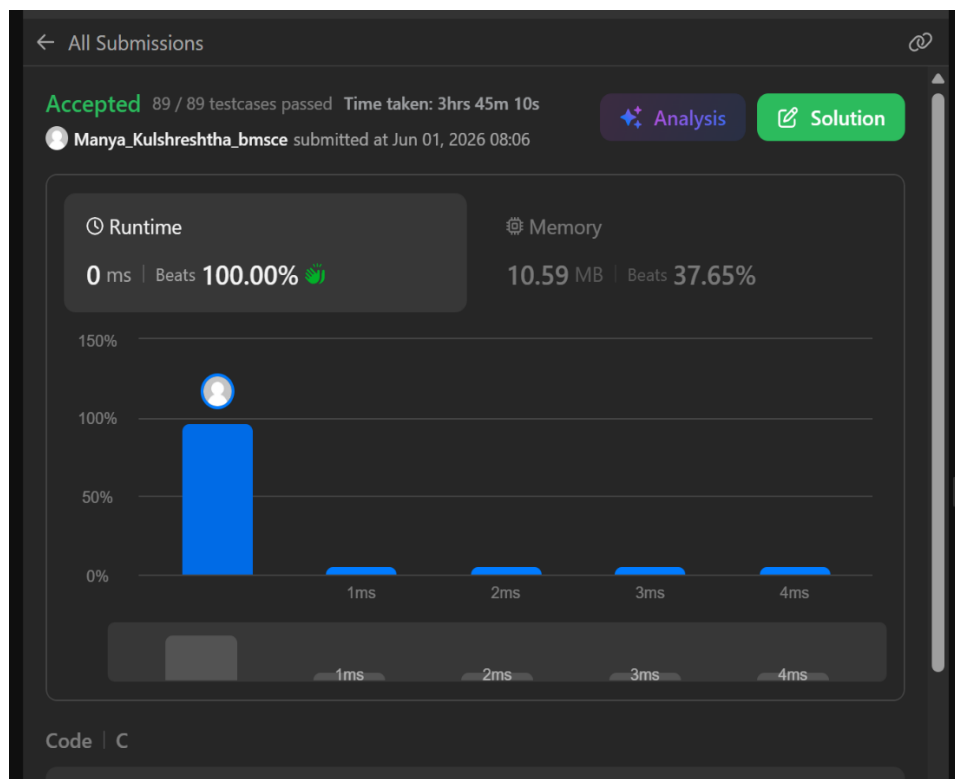


LEETCODE 75. Sort Colors

CODE:

```
void sortColors(int* nums, int numsSize) {  
    int l = 0, m = 0, r = numsSize - 1, t;  
  
    while (m <= r) {  
        if (nums[m] == 0) {  
            t = nums[l];  
            nums[l++] = nums[m];  
            nums[m++] = t;  
        }  
        else if (nums[m] == 2) {  
            t = nums[r];  
            nums[r--] = nums[m];  
            nums[m] = t;  
        }  
        else {  
            m++;  
        }  
    }  
}
```

OUTPUT:



Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[2, 0, 2, 1, 1, 0]

Output

[0, 0, 1, 1, 2, 2]

Expected

[0, 0, 1, 1, 2, 2]

♥ Contribute a

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[2, 0, 1]

Output

[0, 1, 2]

Expected

[0, 1, 2]

♥ Cor

Lab program 5:

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i)
    {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--)
    {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

int main()
{
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1){
        printf("Invalid input.\n");
        return 1;
    }
```

```

    }
    if (n <= 0){
        printf("No elements to sort.\n");
        return 0;
    }
    int *arr = (int *)malloc((size_t)n * sizeof(int));
    if (arr == NULL){
        printf("Memory allocation failed.\n");
        return 1;
    }
    srand(time(0));
    for (int i = 0; i < n; i++){
        arr[i] = rand() % 100000;
    }
    clock_t start, end;
    start = clock();
    heapSort(arr, n);
    end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    printArray(arr, n);
    printf("\n\nTime taken = %f seconds\n", time_taken);
    free(arr);
    return 0;
}

```

OUTPUT:

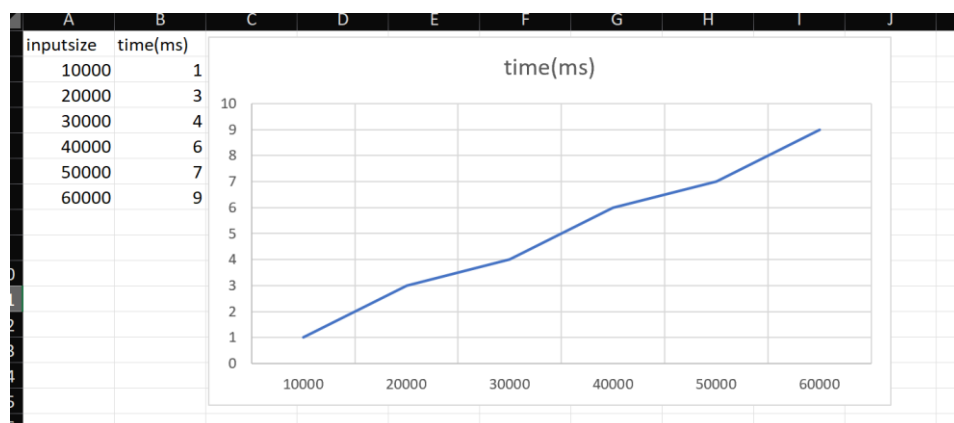
```

Enter number of elements:6
Enter the elements:
5 4 3 6 7 2

Elements after heap sort:2 3 4 5 6 7
Input Size:6 Execution Time:0.00000
Process returned 36 (0x24)   execution time : 6.256 s
Press any key to continue.

```

GRAPH:



Lab program 6:

CODE:

```
#include <stdio.h>
#define MAX 100
int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int W, int wt[], int val[], int n) {
    int dp[MAX][MAX];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            }
            else if (wt[i-1] <= w) {
                dp[i][w] = max(
                    val[i-1] + dp[i-1][w - wt[i-1]],
                    dp[i-1][w]
                );
            }
            else {
                dp[i][w] = dp[i-1][w];
            }
        }
    }
    return dp[n][W];
}

int main() {
    int n, W;
    int wt[MAX], val[MAX];
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter weights:\n");
    for (int i = 0; i < n; i++){
        scanf("%d", &wt[i]);
    }
    printf("Enter values:\n");
    for (int i = 0; i < n; i++){
        scanf("%d", &val[i]);
    }
    printf("Enter capacity of knapsack: ");
    scanf("%d", &W);
    int result = knapsack(W, wt, val, n);
    printf("Maximum profit = %d\n", result);
    return 0;
}
```

OUTPUT:

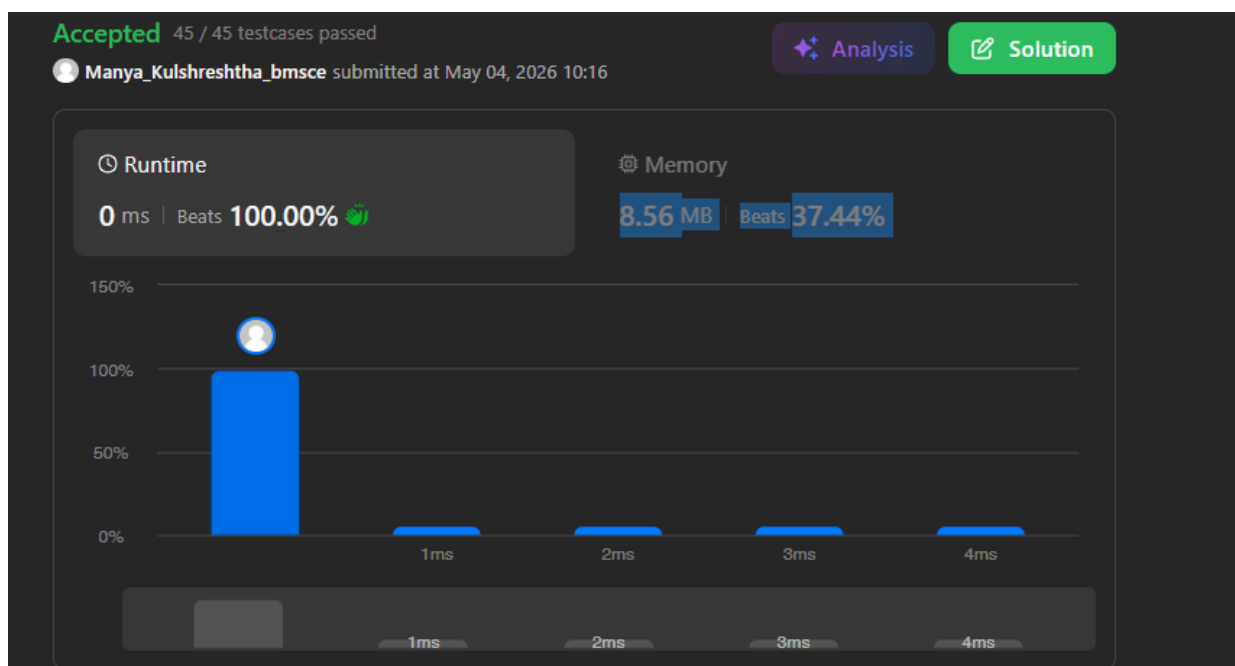
```
Enter number of items: 4
Enter weights:
2
3
4
5
Enter values:
3
4
5
6
Enter capacity of knapsack: 5
Maximum profit = 7
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169\lab7>
```

LEETCODE 70. Climbing Stairs

CODE:

```
int climbStairs(int n) {  
    if (n <= 2) return n;  
  
    int dp[n + 1];  
    dp[1] = 1;  
    dp[2] = 2;  
  
    for (int i = 3; i <= n; i++) {  
        dp[i] = dp[i - 1] + dp[i - 2];  
    }  
  
    return dp[n];  
}
```

OUTPUT:



Lab program 7:

CODE:

```
#include<stdio.h>
#define MAX 100
#define INF 99999
void floydWarshall(int graph[MAX][MAX],int n){
    int dist[MAX][MAX];
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            dist[i][j]=graph[i][j];
        }
    }
    for(int k=0;k<n;k++){
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                if(dist[i][k]+dist[k][j]<dist[i][j]){
                    dist[i][j]=dist[i][k]+dist[k][j];
                }
            }
        }
    }
    printf("All Pairs shortest Path Matrix \n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(dist[i][j]==INF){
                printf("INF ");
            }else{
                printf("%d ",dist[i][j]);
            }
        }
        printf("\n");
    }
}
int main(){
    int n,graph[MAX][MAX];
    printf("Enter no. of Vertices\n");
    scanf("%d",&n);
    printf("Enter adjacency matrix (use %d for INF):\n", INF);
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            scanf("%d", &graph[i][j]);
        }
    }
    floydWarshall(graph, n);
    return 0;
}
```


OUTPUT:

```
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169> cd C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169
Enter no. of Vertices
4
Enter adjacency matrix (use 99999 for INF):
0
5
99999
10
99999
0
3
99999
99999
99999
0
1
99999
99999
99999
0
All Pairs shortest Path Matrix
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169>
```

\

LEETCODE 743. Network Delay Time

CODE:

```
#define INF 1000000000

int networkDelayTime(int** times, int timesSize, int* timesColSize,
                    int n, int k) {

    int graph[101][101];
    int dist[101];
    int visited[101] = {0};

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            graph[i][j] = INF;
        }
    }

    for (int i = 0; i < timesSize; i++) {
        int u = times[i][0];
        int v = times[i][1];
        int w = times[i][2];
        graph[u][v] = w;
    }

    for (int i = 1; i <= n; i++)
        dist[i] = INF;

    dist[k] = 0;

    for (int i = 1; i <= n; i++) {

        int u = -1;

        for (int j = 1; j <= n; j++) {
            if (!visited[j] &&
                (u == -1 || dist[j] < dist[u])) {
                u = j;
            }
        }

        visited[u] = 1;

        for (int v = 1; v <= n; v++) {
            if (graph[u][v] != INF &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
}
```

```

    }
}

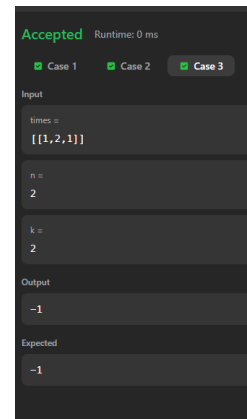
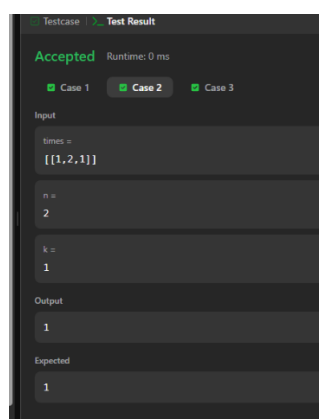
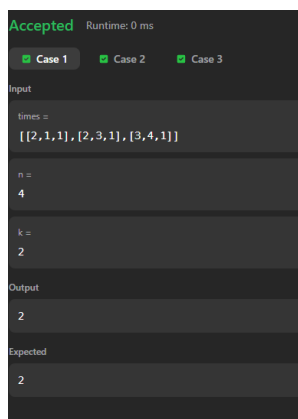
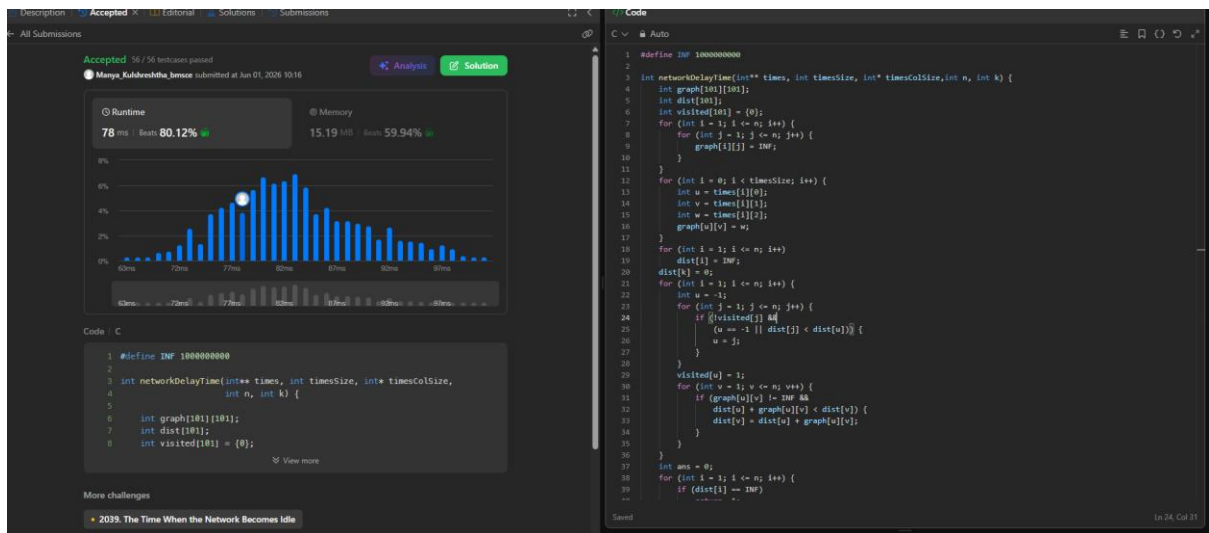
int ans = 0;

for (int i = 1; i <= n; i++) {
    if (dist[i] == INF)
        return -1;

    if (dist[i] > ans)
        ans = dist[i];
}

return ans;
}
OUTPUT:

```



Lab program 8:

8(a)

CODE:

```
#include <stdio.h>
#define V 5
#define INF 99999

int minKey(int key[], int mstSet[])
{
    int min = INF, min_index;
    for (int v = 0; v < V; v++)
    {
        if (mstSet[v] == 0 && key[v] < min)
        {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void printMST(int parent[], int graph[V][V])
{
    int cost = 0;

    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++)
    {
        printf("%d - %d \t%d\n", parent[i], i,
            graph[i][parent[i]]);
        cost += graph[i][parent[i]];
    }

    printf("Total Cost = %d\n", cost);
}

void primMST(int graph[V][V])
{
    int parent[V];
    int key[V];
    int mstSet[V];

    for (int i = 0; i < V; i++)
    {
        key[i] = INF;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++)
    {

```

```

int u = minKey(key, mstSet);
mstSet[u] = 1;

for (int v = 0; v < V; v++)
{
    if (graph[u][v] &&
        mstSet[v] == 0 &&
        graph[u][v] < key[v])
    {
        parent[v] = u;
        key[v] = graph[u][v];
    }
}

printMST(parent, graph);
}

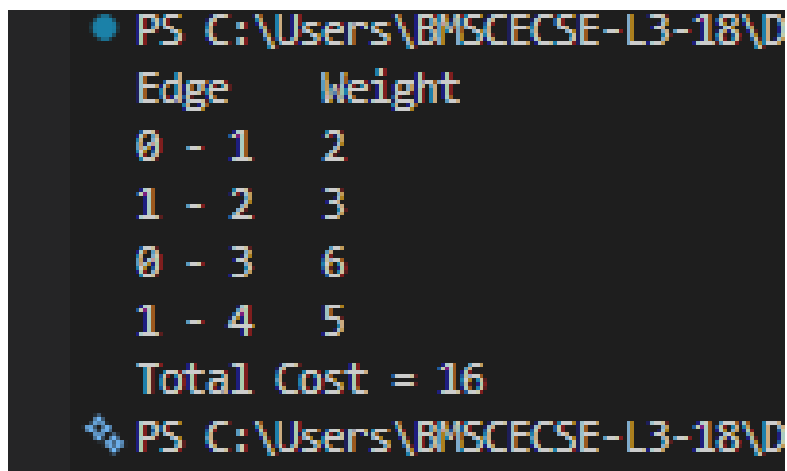
int main()
{
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };

    primMST(graph);

    return 0;
}

```

OUTPUT:



```

PS C:\Users\BMSCECSE-L3-18\Documents>
Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5
Total Cost = 16
PS C:\Users\BMSCECSE-L3-18\Documents>

```

8(b)

CODE:

```
##include <stdio.h>
#include <stdlib.h>

struct Edge
{
    int src, dest, weight;
};

struct Subset
{
    int parent, rank;
};

int find(struct Subset subsets[], int i)
{
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);

    return subsets[i].parent;
}

void Union(struct Subset subsets[], int x, int y)
{
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    if (subsets[xroot].rank < subsets[yroot].rank)
        subsets[xroot].parent = yroot;
    else if (subsets[xroot].rank > subsets[yroot].rank)
        subsets[yroot].parent = xroot;
    else
    {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}

int compare(const void *a, const void *b)
{
    return ((struct Edge *)a)->weight -
        ((struct Edge *)b)->weight;
}

int main()
{
    int V = 4;
    int E = 5;

    struct Edge edges[] = {
        {0, 1, 10},
        {0, 2, 6},
```

```

        {0, 3, 5},
        {1, 3, 15},
        {2, 3, 4}
    };

    qsort(edges, E, sizeof(edges[0]), compare);

    struct Subset subsets[V];

    for (int v = 0; v < V; v++)
    {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }

    printf("Edges in MST:\n");

    int e = 0, i = 0, cost = 0;

    while (e < V - 1)
    {
        struct Edge next_edge = edges[i++];

        int x = find(subsets, next_edge.src);
        int y = find(subsets, next_edge.dest);

        if (x != y)
        {
            printf("%d -- %d == %d\n",
                next_edge.src,
                next_edge.dest,
                next_edge.weight);

            cost += next_edge.weight;
            Union(subsets, x, y);
            e++;
        }
    }

    printf("Total Cost = %d\n", cost);

    return 0;
}

```

OUTPUT:

```
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf
Edges in MST:
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
Total Cost = 19
❖ PS C:\Users\BMSCECSE-L3-18\Desktop\1bf
```


Lab program 9:

CODE:

```
#include<stdio.h>
#include<time.h>
struct Item
{
    int weight;
    int profit;
    float ratio;
};
void swap(struct Item *a , struct Item *b){
    struct Item temp=*a ;
    *a=*b;
    *b=temp;
};
void SortbyRatio(struct Item arr[],int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(arr[j].ratio<arr[j+1].ratio){
                swap(&arr[j],&arr[j+1]);
            }
        }
    }
}
void SortbyProfit(struct Item arr[],int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(arr[j].profit<arr[j+1].profit){
                swap(&arr[j],&arr[j+1]);
            }
        }
    }
}
void SortbyWeight(struct Item arr[],int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(arr[j].weight<arr[j+1].weight){
                swap(&arr[j],&arr[j+1]);
            }
        }
    }
}
float knapsack(struct Item arr[],int n ,int capacity){
    float totalProfit=0;
    for(int i=0;i<n;i++){
        if(capacity>=arr[i].weight){
            totalProfit+=arr[i].profit;
            capacity-=arr[i].weight;
        }else{
            totalProfit+=arr[i].ratio*capacity;
            break;
        }
    }
}
```

```

    }
    return totalProfit;
}

int main(){
    int n , capacity, choice;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item arr[n];
    printf("Enter weights:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i].weight);
    }

    printf("Enter profits:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i].profit);
        arr[i].ratio = (float)arr[i].profit / arr[i].weight;
    }

    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    printf("choose a.p/w (optimal) b. profit c,weight\n");
    scanf("%d",&choice);
    clock_t start = clock();
    switch(choice){
        case 1: SortbyRatio(arr,n);
        printf("Optimal method\n");
        break;
        case 2: SortbyProfit(arr,n);
        printf("By profit\n");
        break;
        case 3: SortbyWeight(arr,n);
        printf("By weight \n");
        break;
        default: printf("Invalid choice\n");
        break;
    }
    float maxProfit=knapsack(arr, n, capacity);
    clock_t end=clock();
    double time_taken=((double)(end-start))/CLOCKS_PER_SEC;
    printf("Max Profit: %.2f\n",maxProfit);
    printf("execution time : %f seconds \n",time_taken);
    return 0;
}

```

OUTPUT:

```

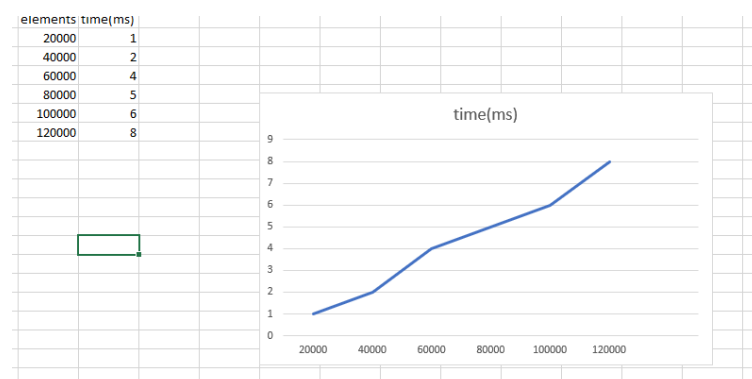
Enter number of items: 4
Enter weights:
10
20
40
30
Enter profits:
100
120
90
170
Enter knapsack capacity: 50
choose a.p/w (optimal) b. profit c,weight
1
Optimal method
Max Profit: 333.33

```

```

Enter number of items: 4
Enter weights:
10
20
30
40
Enter profits:
100
120
90
170
Enter knapsack capacity: 50
choose a.p/w (optimal) b. profit c,weight
2
By profit
Max Profit: 230.00
execution time : 0.000000 seconds
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169> ./knapsack.exe
Enter number of items: 10
Enter weights:
2
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169> ./knapsack.exe
Enter number of items: 4
Enter weights:
10
20
30
40
Enter profits:
100
120
90
170
Enter knapsack capacity: 50
choose a.p/w (optimal) b. profit c,weight
3
By weight
Max Profit: 200.00
execution time : 0.000000 seconds
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169>

```

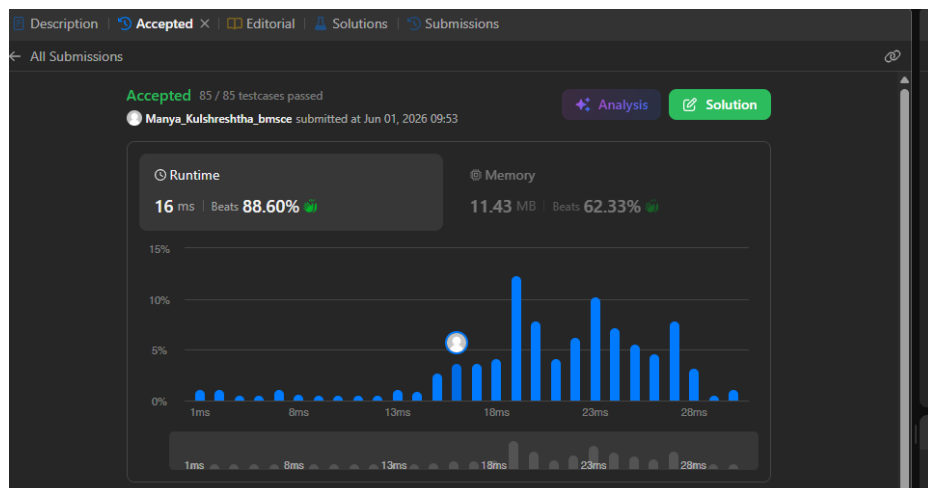


LEETCODE 561. Array Partition

CODE:

```
int compare(const void *a, const void *b) {  
    return (*(int *)a - *(int *)b);  
}  
  
int arrayPairSum(int* nums, int numsSize) {  
    qsort(nums, numsSize, sizeof(int), compare);  
  
    int sum = 0;  
    for (int i = 0; i < numsSize; i += 2)  
        sum += nums[i];  
  
    return sum;  
}
```

OUTPUT:



Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[1,4,3,2]

Output

4

Expected

4

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[6,2,6,5,1,2]

Output

9

Expected

9

Lab program 10:

CODE:

```
#include<stdio.h>
#include<limits.h>
#define MAX 100
int minDistance(int dist[], int visited[],int n){
    int min= INT_MAX, min_index;
    for(int i=0;i<n;i++){
        if(visited[i]==0 && dist[i]<=min){
            min=dist[i];
            min_index=i;
        }
    }
    return min_index;
}
void dijkstra(int graph[MAX][MAX],int n, int src){
    int dist[MAX],visited[MAX];
    for(int i=0;i<n;i++){
        dist[i]=INT_MAX;
        visited[i]=0;
    }
    dist[src]=0;
    for(int count=0;count<n-1;count++){
        int u=minDistance(dist,visited,n);
        visited[u]=1;
        for(int v=0;v<n;v++){
            if(!visited[v]&& graph[u][v]&& dist[u]!=INT_MAX &&
dist[u]+graph[u][v]<dist[v]){
                dist[v]=dist[u]+graph[u][v];
            }
        }
    }
    printf("Vertex \t Dist from src\n");
    for(int i=0;i<n;i++){
        printf("%d\t%d\n",i,dist[i]);
    }
}
int main() {
    int n, graph[MAX][MAX],src;
    printf("Enter no of vertices\n");
    scanf("%d",&n);
    printf("Enter adj matrix\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            scanf("%d",&graph[i][j]);
        }
    }
    printf("enter src vertex\n");
    scanf("%d",&src);
```

```
dijkstra(graph,n,src);  
return 0;  
}
```

OUTPUT:

```
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169> cd "c:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169\lab7"
Enter no of vertices
5
Enter adj matrix
0
10
0
5
0
0
0
1
2
0
0
0
0
0
4
0
3
9
0
2
7
0
0
0
0
enter src vertex
0
Vertex  Dist from src
0      0
1      8
2      9
3      5
4      7
PS C:\Users\BMSCECSE-L3-18\Desktop\1bf24cs169\lab7>
```

Lab program 11:

CODE:

```
#include <stdio.h>

int board[20], n, count = 0;

int isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col ||
            board[i] - i == col - row ||
            board[i] + i == col + row)
            return 0;
    }
    return 1;
}

void solve(int row) {
    if (row == n) {
        count++;
        printf("\nSolution %d:\n", count);

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i] == j)
                    printf("Q ");
                else
                    printf(". ");
            }
            printf("\n");
        }
        return;
    }

    for (int col = 0; col < n; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            solve(row + 1);
        }
    }
}

int main() {
    printf("Enter N: ");
    scanf("%d", &n);

    solve(0);

    printf("\nTotal Solutions = %d\n", count);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\BMSCECSE-L3-18\Documents>
Enter N: 4

Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .
```